Omnicron OMI-Lisp Runtime Closure

Final Architectural Lock

1. Core declaration

The Omnicron OMI-Lisp variant is a portable and embeddable declarative runtime engine.

It does not depend on a privileged fixed 8-tuple as its object model.

Instead, its environment emerges dynamically from the interaction of:

```
FS / GS / RS / US
scope coordinates

CAR / CDR
ordered structural coordinates

APVAL / APVAL1
value-binding classes

SUBR / FSUBR
system operator classes

EXPR / FEXPR
user operator classes

SEXPR
canonical structural notation

MEXPR
human-readable operator notation

CONS
ordered resolver, coproduct mediator,
and tetrahedral centroid relation
```

The complete runtime is therefore:

```
scope
×
binding class
×
```

```
local coordinate
×
origin-tagged contribution
×
resolver profile
```

It is generated from typed relations rather than imposed as one static tuple.

---

2. Final deprecation of the 8-tuple authority

The historical 8-tuple may remain in documentation as:

```
a mnemonic

a pedagogical diagram

a migration reference

an explanatory coordinate surface
```

It MUST NOT remain the normative runtime object.

It MUST NOT determine:

```
environment layout

scope identity

binding dispatch

CONS resolution

blackboard composition

character reachability

validation

serialization

projection
```

Canonical deprecation rule:

```
The 8-tuple is non-authoritative explanatory notation.

The Omnicron Object Model is the authoritative
dynamic runtime structure.
```

The runtime no longer asks:

```
Which fixed tuple position owns this object?
```

It asks:

```
Which scope contains it?

Which binding class defines it?

Which source contributed it?

Which CAR/CDR relation addresses it?

Which resolver composes it?

Which character coordinate makes it readable?
```

---

3. The emergent environment

For each symbol s, each scope may expose an environment entry:

$$E_\sigma(s)$$

where:

$$\sigma \in \{FS, GS, RS, US\}$$

and:

$$E_\sigma(s) = [CAR, CDR, APVAL, APVAL1, SUBR, FSUBR, EXPR, FEXPR]$$

The complete symbol environment is:

$$E(s) = [E_{FS}(s) : E_{GS}(s) : E_{RS}(s) : E_{US}(s)]$$

This forms a dynamic:

$$4 \times 8 = 32$$

typed environment surface.

The four scopes determine:

```
where a binding applies
```

The eight environment classes determine:

```
what kind of binding it is
```

No fixed tuple must be interpreted globally before the symbol can be resolved.

---

4. Environment classes

The canonical Omicron low-gauge assignment is:

```
0x00..0x1A
CAR

0x1B..0x1F
CDR

0x20..0x3A
APVAL

0x3B..0x3F
APVAL1

0x40..0x5A
SUBR

0x5B..0x5F
FSUBR

0x60..0x7A
EXPR
```

```
0x7B..0x7F
FEXPR
```

Their responsibilities are:

```
CAR
carried structural coordinate

CDR
continuation or resolver coordinate

APVAL
primary data binding

APVAL1
alternate or scoped data binding

SUBR
system ordinary function

FSUBR
system special form

EXPR
user-defined ordinary function

FEXPR
user-defined special form
```

Ordinary functions evaluate operands before invocation:

```
SUBR
EXPR
```

Special forms receive their operand forms unevaluated:

```
FSUBR
FEXPR
```

---

5. Notation surfaces

The runtime has two complementary notation surfaces.

SEXPR

The canonical structural representation is the S-expression.

```
(projective
  (FS . (CAR_FS . CDR_FS))
  (GS . (CAR_GS . CDR_GS))
  (RS . (CAR_RS . CDR_RS))
  (US . (CAR_US . CDR_US)))
```

SEXPR preserves:

```
ordered pairs

types

scope tags

nesting

source identity

resolver identity

canonical serialization
```

MEXPR

The M-expression is the human-facing operator surface.

```
PROJECTIVE[
  CONS_FS(CAR_FS,CDR_FS) :
  CONS_GS(CAR_GS,CDR_GS) :
  CONS_RS(CAR_RS,CDR_RS) :
  CONS_US(CAR_US,CDR_US)
]
```

Every M-expression MUST lower deterministically to one canonical S-expression.

Canonical boundary:

```
MEXPR presents.

SEXPR preserves.

The evaluator executes the lowered structure.
```

---

6. Projective blackboard coordinate

Each scope resolves one ordered CONS relation:

$$X_{FS} = CONS_{FS}(CAR_{FS}, CDR_{FS})$$

$$X_{GS} = CONS_{GS}(CAR_{GS}, CDR_{GS})$$

$$X_{RS} = CONS_{RS}(CAR_{RS}, CDR_{RS})$$

$$X_{US} = CONS_{US}(CAR_{US}, CDR_{US})$$

The complete homogeneous tetrahedral coordinate is:

$$P = [X_{FS} : X_{GS} : X_{RS} : X_{US}]$$

or:

```
P =
[CONS_FS : CONS_GS : CONS_RS : CONS_US]
```

This is the primary unresolved blackboard relation.

The four values are not fields in a historical tuple. They are independently produced scoped relations.

---

7. CONS as emergent centroid

CONS is not:

```
a fifth scope

a fifth QuQuart state
```

```
a fixed tuple position

a global mutable register
```

CONS is the relation induced by the active scoped contributions.

It may act as:

```
ordered CAR/CDR resolver

quasigroup recovery operation

coproduct mediating map

sparse-board compositor

tetrahedral centroid reducer

affine-chart normalizer

missing-coordinate recovery witness
```

Therefore:

```
CONS is emergent from the object relations.

CONS is not preallocated as an additional basis object.
```

The tetrahedral QuQuart model already establishes CONS as the centroid rather than a fifth basis state and distinguishes Boolean capacity from reversible quasigroup resolution.

---

8. Coproduct blackboard

Independent local or remote `.o` sources contribute tagged boards:

$$B_i \subseteq \{0, \dots, 255\}$$

The authoritative blackboard begins as:

$$\mathcal{B} = \bigsqcup_{i \in I} B_i$$

Each element preserves:

```
origin

scope

CAR

CDR

binding class

resolver profile

board coordinate

version
```

CONS maps the origin-tagged coproduct into the shared visible plane:

$$CONS : \bigsqcup_i B_i \to P_{256}$$

The shared plane is:

$$P_{256} = \{0, \ldots, 255\}$$

A visible byte is only the projection.

Its fiber retains all full contributions behind it.

---

9. Sparse 256-bit contribution model

Each `.o` source contributes one selected rank-8 predicate:

```
256 possible input coordinates

one output bit for each coordinate

256 bits total

32 bytes of storage
```

The runtime does not instantiate the complete population of 2^{256} possible predicates.

It stores only the selected board.

```
typedef struct {
    uint64_t lane[4];
} OmiBoard256;
```

Potentially remote contributors may therefore submit bounded, finite, independently verifiable knowledge boards without sharing one mutable global heap.

---

10. Shared OMI-Lisp plane

CONS composes compatible boards into one 256-position visible plane while retaining the complete ordered relation:

```
CAR = 32-bit omi---imo coordinate

CDR = 32-bit ΔUS/?O_oΔ continuation

CONS = resolved relation

plane coordinate = bounded visible projection
```

The visible byte MUST NOT replace:

```
CAR

CDR

source identity

scope

resolver profile

binding class
```

Canonical law:

```
The byte is the shared view.

The ordered relation is the identity.
```

---

11. Earned character encoding

The OMI-Lisp character surface is not assumed in advance.

The low Omicron gauge first establishes:

```
hidden control reachability

FS/GS/RS/US scope placement

US tangent

SP crossing

printable branch reachability
```

The low character doctrine defines `US` at `0x1F` as the hidden tangent, `SP` at `0x20` as the readable rupture, and the F-column as the readable branch through `/ ? O _ o`.

The runtime sequence is:

```
source contribution

→ origin-tagged injection

→ scope placement

→ environment-class selection

→ control-plane traversal

→ SP crossing

→ character reachability

→ SEXPR parsing or MEXPR lowering
```

```
→ CONS resolution

→ shared-plane projection
```

The character is therefore earned as the readable coordinate of a lawful runtime relation.

---

12. Locked low/high carrier geometry

The complete byte plane remains divided by the exact involution:

$$mirror(x) = x \oplus 0x80$$

The four locked regions are:

```
0x00..0x1F
low affine-control coordinate field

0x20..0x7F
low affine readable space

0x80..0x9F
high projective-control mirror field

0xA0..0xFF
high projective readable space
```

This carrier geometry is not replaced by the environment decomposition.

The same byte field admits several compatible views:

```
low/high affine-projective mirror

four 32-position character sectors

eight 27+5 environment bands

four scopes × eight environment classes

256-position sparse blackboard
```

Each decomposition answers a different runtime question.

---

13. Dynamic dispatch

When a symbol appears in value position, the runtime searches:

```
APVAL

APVAL1
```

according to the current scope and binding profile.

When a symbol appears in operator position, the runtime searches:

```
SUBR

FSUBR

EXPR

FEXPR
```

The selected binding determines:

```
system or user authority

ordinary or special evaluation

operand evaluation policy

capability boundary
```

Dispatch is therefore a function of:

$$Dispatch(symbol, scope, position, environment)$$

not a fixed tuple lookup.

---

14. Portability

The runtime is portable because its normative core requires only:

```
fixed-width integers

tagged values

ordered CONS cells

bounded bitboards

deterministic parser

deterministic evaluator

explicit environment tables

registered host callbacks
```

It does not require:

```
one operating system

one filesystem

one graphical environment

one network stack

one machine word width beyond declared profiles

one global mutable symbol table
```

The same core may run as:

```
embedded C library

command-line evaluator

WASM module

microcontroller runtime

host application extension

server-side resolver
```

```
offline local blackboard

peer-to-peer module
```

---

15. Embeddability

A host embeds the engine by providing bounded capabilities.

```
typedef struct {
    void *context;

    OmiStatus (*read_module)(
        void *context,
        OmiModuleId id,
        OmiSlice *out
    );

    OmiStatus (*write_projection)(
        void *context,
        const OmiProjection *projection
    );

    OmiStatus (*resolve_remote)(
        void *context,
        const OmiRemoteRequest *request,
        OmiRemoteResult *out
    );
} OmiHost;
```

The core does not grant ambient authority.

A module receives only the capabilities explicitly injected into its environment.

---

16. Declarative runtime character

OMI-Lisp is declarative because a form specifies:

```
which relation exists

which scope contains it
```

```
which binding class defines it

which continuation resolves it

which board positions it activates

which projection represents it
```

The declaration does not require the source to own the evaluator or mutate the host directly.

Effects remain host-mediated and capability-bounded.

---

17. Dynamic completeness

The runtime is dynamically complete in the architectural sense because it contains defined mechanisms for:

```
data binding

alternate data binding

system functions

system special forms

user functions

user special forms

ordered structure

continuation

scope

remote contribution

sparse blackboard composition

reversible resolution

canonical notation

human-readable notation
```

```
projection

validation boundaries

extension
```

“Complete” here does not mean that every mathematical or programming operation is built in.

It means the runtime has a closed, extensible account of how new operations and values enter, resolve, and remain typed.

---

18. Extension without tuple expansion

A new operation does not require adding another global tuple position.

It enters through:

```
an origin-tagged module

a scope

an environment class

a local coordinate

a resolver profile

a capability declaration
```

For example:

```
(inject
  (origin . geometry.o)
  (scope . GS)
  (class . EXPR)
  (symbol . project-centroid)
  (value . ...))
```

A system special form may enter as:

```
(inject
  (origin . runtime.o)
  (scope . FS)
  (class . FSUBR)
  (symbol . projective)
  (value . ...))
```

The object model grows by coproduct injection, not by changing a fixed global tuple.

---

19. Authority boundaries

```
The character gauge establishes reachability.

The parser establishes syntax.

The environment establishes binding.

The evaluator establishes execution.

CONS establishes local resolution.

The coproduct preserves origin.

The blackboard exposes shared projection.

Validation establishes admissibility.

Projection does not create authority.
```

No one layer silently inherits the authority of another.

---

20. Final architectural identity

The closed architecture is:

```
Omnicron Object Model
=
scoped tagged environments
+
origin-preserving coproduct
```

```
+
sparse 256-bit knowledge boards
+
ordered CAR/CDR relations
+
CONS centroid resolution
+
earned OMI-Lisp notation
+
deterministic SEXPR/MEXPR lowering
+
portable host capability interface
```

The former 8-tuple is unnecessary because the runtime can construct every active coordinate from declared objects and relations.

---

21. Canonical one-line statement

> The Omnicron OMI-Lisp variant is a portable and embeddable declarative runtime engine whose environment emerges dynamically from four FS/GS/RS/US scope coordinates crossed with eight CAR/CDR/APVAL/APVAL1/SUBR/FSUBR/EXPR/FEXPR binding classes; independent local or remote `.o` modules enter through an origin-preserving coproduct of sparse 256-bit boards, CONS resolves their ordered 32-bit CAR/CDR relations into a shared 256-position character plane without erasing provenance, and readable S-expression or M-expression notation is earned only through the Omicron character gauge rather than imposed by a fixed historical 8-tuple.

---

22. Final lock

```
The Omnicron OMI-Lisp runtime is object-based,
not tuple-based.
```

```
The eight environment classes are runtime binding classes,
not a replacement pedagogical tuple.
```

```
FS/GS/RS/US provide scope.

The environment classes provide binding type.

The source tag provides origin.
```

```
CAR/CDR provide ordered relation.

CONS provides resolution.
```

```
The runtime environment emerges
from the active objects and relations.
```

```
New capabilities enter by typed coproduct injection.

They do not require expansion of a global tuple.
```

```
SEXPR is canonical structure.

MEXPR is readable surface syntax.
```

```
Independent .o sources contribute
bounded sparse boards.
```

```
CONS composes the shared plane
without collapsing full identity.
```

```
The OMI-Lisp character is earned
through lawful gauge reachability.
```

```
The 8-tuple is deprecated with finality
as a normative architecture object.
```

```
The Omnicron Object Model is now
the complete portable declarative runtime foundation.
```